Experiment 5: Bias–Variance, Overfitting and Underfitting Analysis

Aim

To fit polynomial regression models of degrees 1, 3, 9 and 15 to synthetic house-price data, compare training and validation errors, estimate bias^2 and variance through repeated resampling, and select an appropriate model.

Mathematical Formulae

1. Polynomial regression

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_d x^d$$

2. Mean squared error

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

3. Bias squared

$$Bias^2(x) = [E(\hat{f}(x)) - f(x)]^2$$

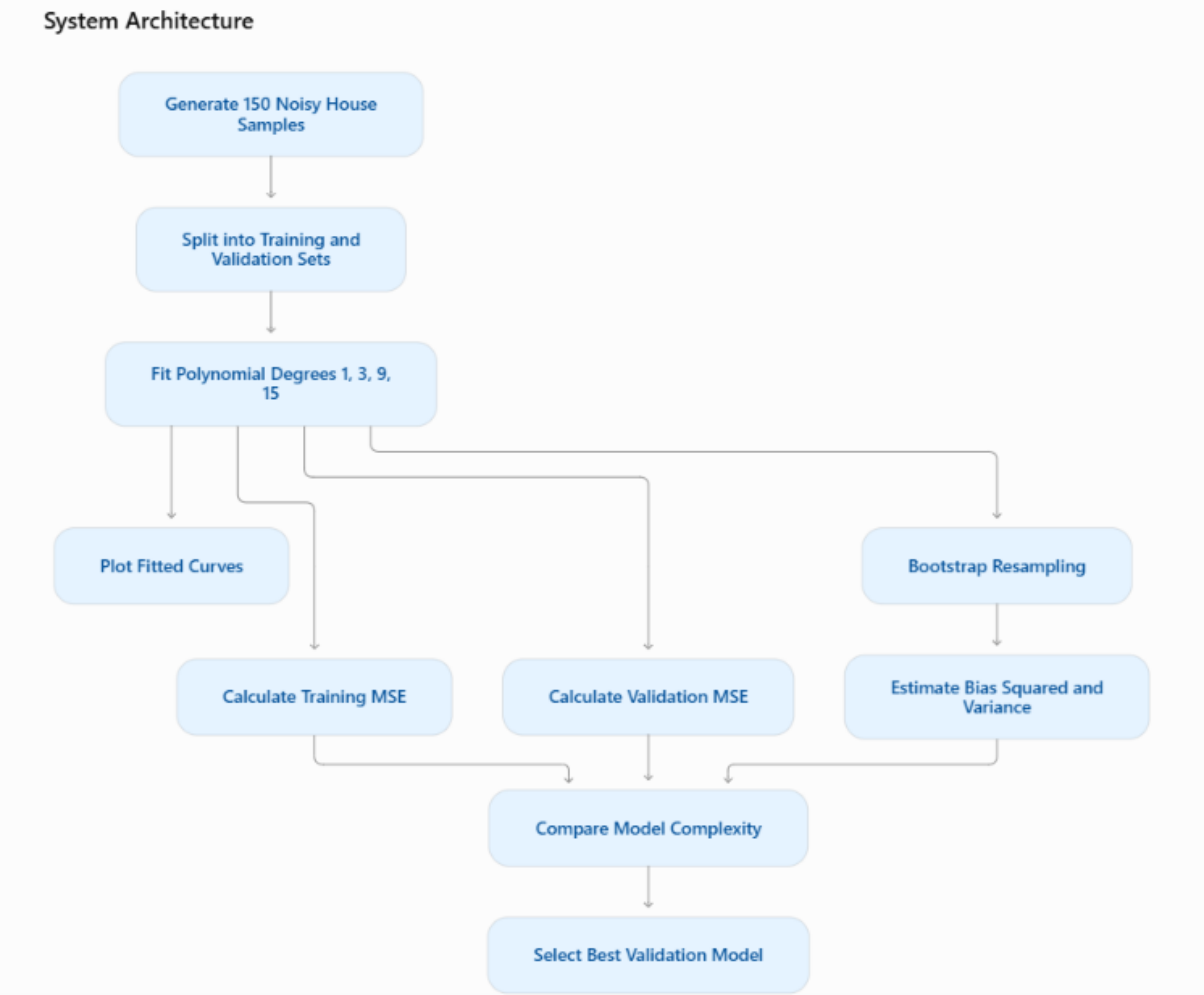
4. Variance

$$Variance(x) = E[(\hat{f}(x) - E(\hat{f}(x)))^2]$$

5. Bias–variance decomposition

$$E[(\hat{f}(x) - y)^2] = Bias^2(x) + Variance(x) + \sigma^2$$

Here, σ^2 represents irreducible noise under the standard squared-error assumptions.



Algorithm / Procedure

1. Generate 150 noisy samples representing house area and price.
2. Split the data into training and validation sets.
3. Fit polynomial regression models of degrees 1, 3, 9 and 15.
4. Plot all fitted curves over the data.
5. Calculate training and validation mean squared errors.
6. Repeatedly resample the training data with replacement.
7. Estimate bias^2 and variance from the bootstrap predictions.
8. Select the model with the lowest validation error and justify the choice.

Python Program

```
import numpy as np

import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

np.random.seed(42)

# 1. Generate 150 noisy house-price samples
n = 150

area = np.sort(np.random.uniform(500, 3000, n))
X = area.reshape(-1, 1)

# Hidden non-linear relationship
def true_function(x):
    return 50 + 0.08 * x + 0.000015 * x**2

price = true_function(area) + np.random.normal(0, 25, n)
y = price

print("Number of Samples:", len(X))
print("First 5 Areas:", X[:5].flatten())
print("First 5 Prices:", y[:5])

# Split into training and validation sets
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.25, random_state=42
)

degrees = [1, 3, 9, 15]
models = {}
```

```
# 2. Fit polynomial regression models
x_plot = np.linspace(500, 3000, 400).reshape(-1, 1)

plt.figure(figsize=(10, 6))
plt.scatter(X_train, y_train, alpha=0.6, label="Training Data")
plt.scatter(X_val, y_val, alpha=0.6, label="Validation Data")

for degree in degrees:
  model = make_pipeline(
    PolynomialFeatures(degree=degree, include_bias=False),
    StandardScaler(),
    LinearRegression()
  )

  model.fit(X_train, y_train)
  models[degree] = model

  predicted_curve = model.predict(x_plot)

  plt.plot(
    x_plot, predicted_curve,
    label=f"Degree {degree}"
  )

plt.title("Polynomial Regression: Model Complexity")
plt.xlabel("House Area")
plt.ylabel("House Price")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# 3. Training and validation errors
train_errors = []
```

```
val_errors = []

print("\nModel Error Comparison")

for degree, model in models.items():
  train_pred = model.predict(X_train)
  val_pred = model.predict(X_val)

  train_mse = mean_squared_error(y_train, train_pred)
  val_mse = mean_squared_error(y_val, val_pred)

  train_errors.append(train_mse)
  val_errors.append(val_mse)

  print(f"\nDegree {degree}")
  print("Training MSE:", round(train_mse, 2))
  print("Validation MSE:", round(val_mse, 2))

plt.figure(figsize=(8, 5))
plt.plot(degrees, train_errors, marker="o",
      label="Training Error")
plt.plot(degrees, val_errors, marker="s",
      label="Validation Error")

plt.xlabel("Polynomial Degree")
plt.ylabel("Mean Squared Error")
plt.title("Training vs Validation Error")
plt.xticks(degrees)
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# 4. Estimate bias squared and variance by bootstrap resampling
```

```
# Use a fixed grid to compare predictions with the true function.
X_eval = np.linspace(500, 3000, 150).reshape(-1, 1)
y_true = true_function(X_eval.ravel())

n_bootstrap = 100
bias_squared = []
variances = []

for degree in degrees:
  bootstrap_predictions = []

  for _ in range(n_bootstrap):
    # Sample training records with replacement
    indices = np.random.choice(
      len(X_train), size=len(X_train), replace=True
    )

    X_boot = X_train[indices]
    y_boot = y_train[indices]

    model = make_pipeline(
      PolynomialFeatures(degree=degree, include_bias=False),
      StandardScaler(),
      LinearRegression()
    )

    model.fit(X_boot, y_boot)
    predictions = model.predict(X_eval)
    bootstrap_predictions.append(predictions)

  predictions = np.array(bootstrap_predictions)

  # Average prediction across bootstrap models
  mean_prediction = predictions.mean(axis=0)
```

```
    # Bias squared and variance averaged over evaluation points
    bias2 = np.mean((mean_prediction - y_true) ** 2)
    variance = np.mean(np.var(predictions, axis=0))

    bias_squared.append(bias2)
    variances.append(variance)

    print(f"\nDegree {degree}")
    print("Estimated Bias^2:", round(bias2, 2))
    print("Estimated Variance:", round(variance, 2))

# Plot bias squared and variance
plt.figure(figsize=(8, 5))
plt.plot(degrees, bias_squared, marker="o",
        label="Bias Squared")
plt.plot(degrees, variances, marker="s",
        label="Variance")

plt.xlabel("Polynomial Degree")
plt.ylabel("Estimated Error Component")
plt.title("Bias-Variance Analysis")
plt.xticks(degrees)
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# 5. Select the degree with the lowest validation MSE
best_index = np.argmin(val_errors)
best_degree = degrees[best_index]

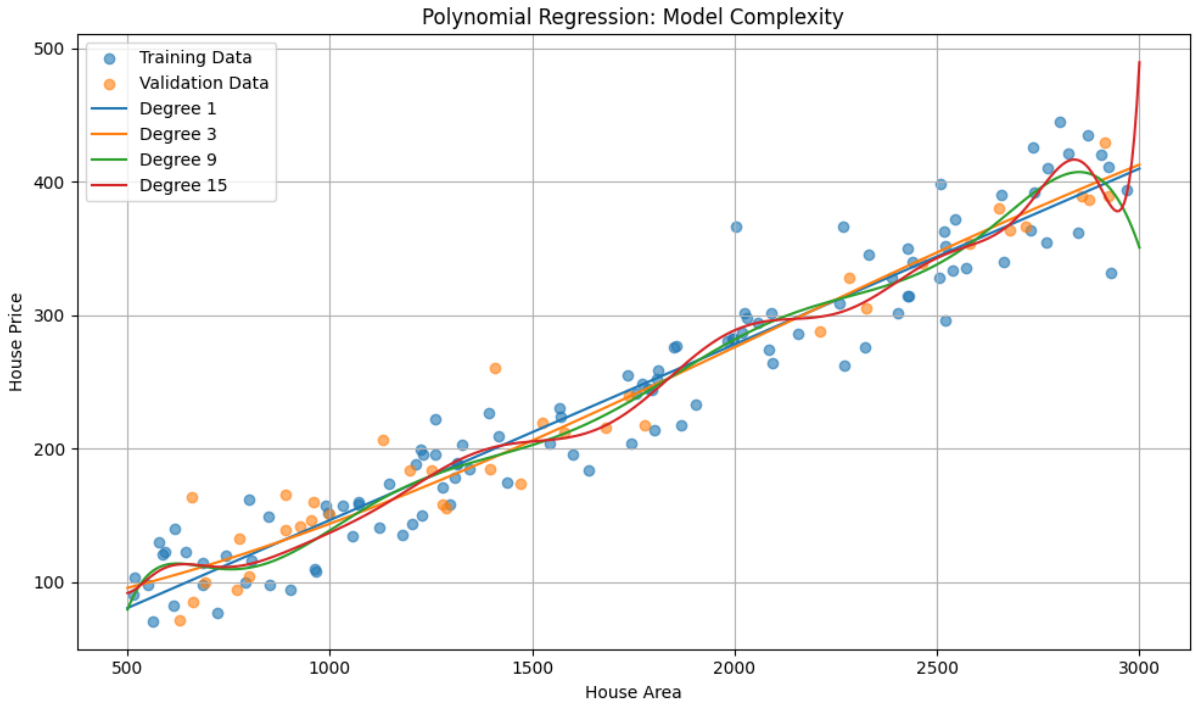
print("\nBest Degree:", best_degree)
print("Lowest Validation MSE:",
```

```
    round(val_errors[best_index], 2))


if best_degree == 1:
    print("Conclusion: The linear model performed best.")
elif best_degree == 3:
    print("Conclusion: Degree 3 provides the best validation performance.")
else:
    print("Conclusion: A higher-degree model performed best on validation data.")
```

Output

```
Number of Samples: 150
First 5 Areas: [513.80529281 517.38032633 551.46123574 563.54781686 578.57296422]
First 5 Prices: [ 90.25533749 102.9443457   97.81074703  70.63066654 129.87760762]
```



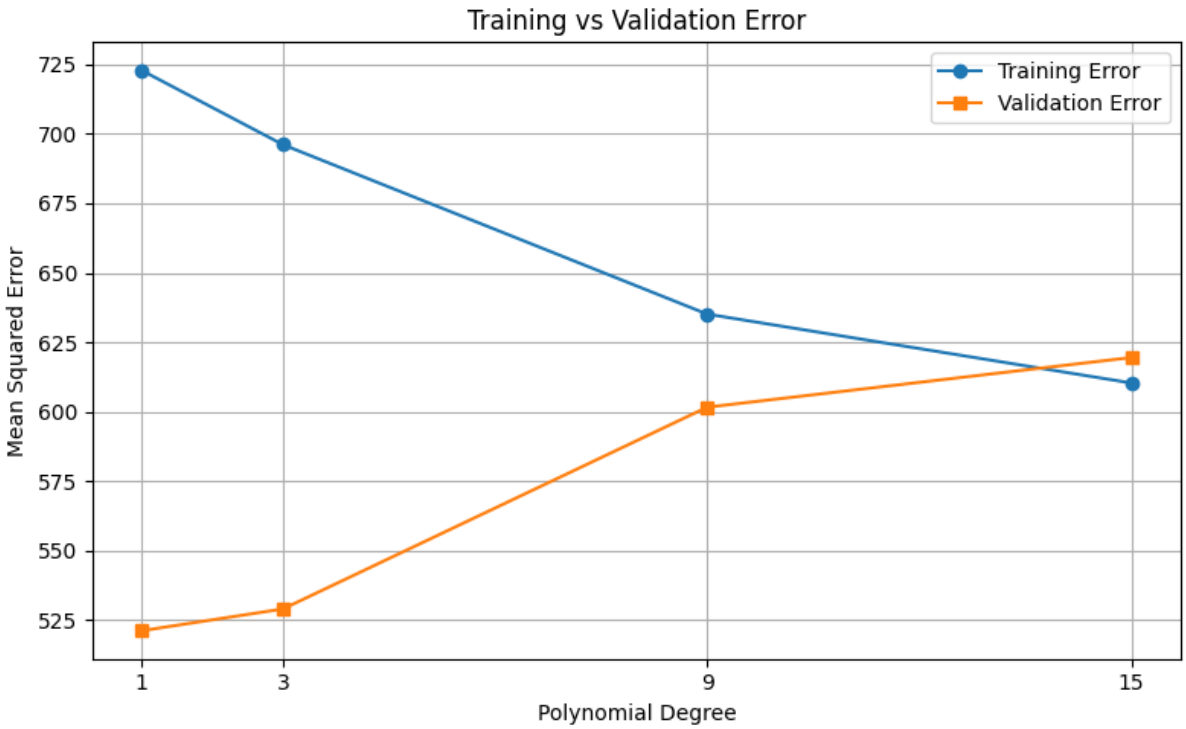

```
Model Error Comparison

Degree 1
Training MSE: 722.87
Validation MSE: 521.14

Degree 3
Training MSE: 696.12
Validation MSE: 528.98

Degree 9
Training MSE: 635.12
Validation MSE: 601.6

Degree 15
Training MSE: 610.33
Validation MSE: 619.48
```

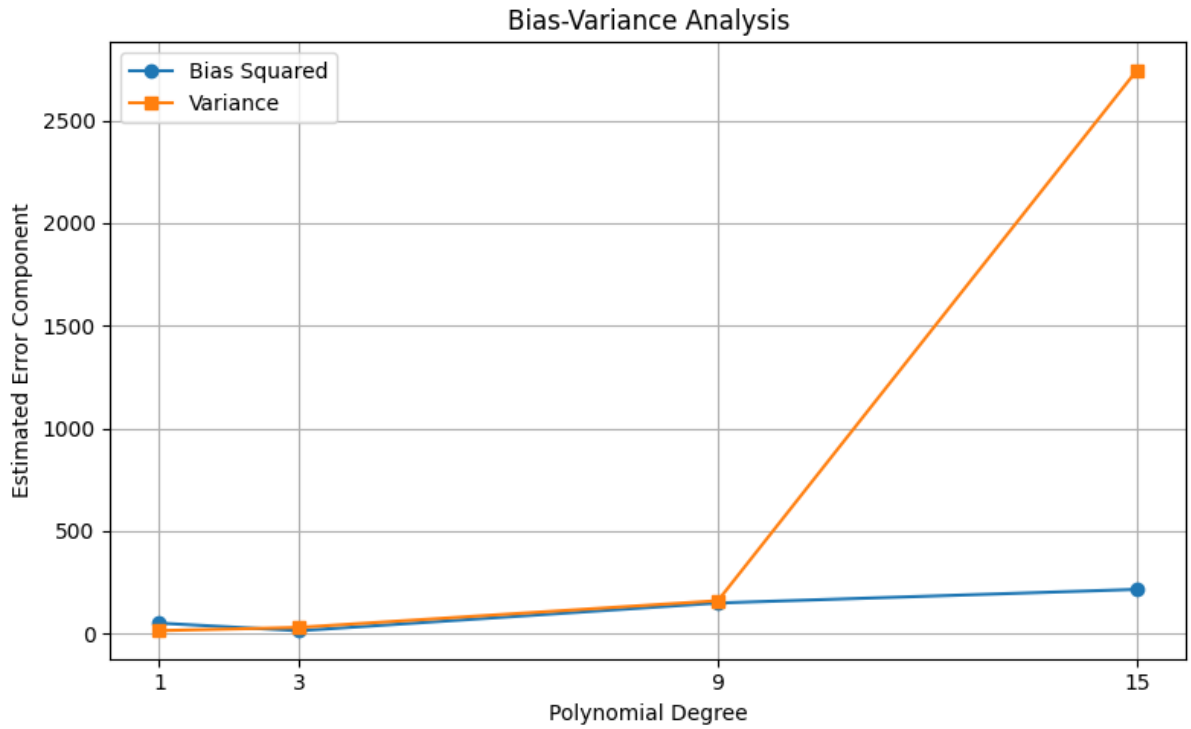



```
Degree 1
Estimated Bias^2: 51.99
Estimated Variance: 15.21

Degree 3
Estimated Bias^2: 14.69
Estimated Variance: 30.95

Degree 9
Estimated Bias^2: 149.84
Estimated Variance: 160.87

Degree 15
Estimated Bias^2: 216.27
Estimated Variance: 2743.96
```




```
Best Degree: 1
Lowest Validation MSE: 521.14
Conclusion: The linear model performed best.
```

Result

Thus, polynomial regression models of different complexities were fitted and evaluated. Training error, validation error, bias^2 and variance were compared to study underfitting and overfitting. The model with the lowest validation MSE was selected as the preferred model for the generated dataset.